Unité 9 — Bonnes pratiques & SAST

Livrable B — Support de cours · UE « Software Security »

Code UE 7532 14 U32 D1 · 11 octobre 2026

> **Place dans le programme.** « Évaluer les bonnes pratiques et mécanismes de défense » et « évaluer la sécurité d'un code source ». On passe du curatif (exploiter/corriger) au **préventif** : programmation défensive et détection précoce par **analyse statique (SAST)**. *Programmation sûre (OpenBSD, libsafe) évoquée dans meltypot ; détails de programmation défensive et outils SAST concrets (bandit, cppcheck, semgrep) = contenu auteur.*

1. Programmation défensive

Principe	Application
Valider les entrées	Liste blanche de formats/longueurs ; rejeter par défaut.
Séparer code et données	Requêtes paramétrées, listes d'arguments (pas de shell) — cf. U7/U8.
Encoder les sorties	Selon le contexte (HTML, SQL, shell).
Moindre privilège	Comptes/jetons au minimum nécessaire ; cloisonnement.
Fail-safe	En cas d'erreur, refuser plutôt qu'autoriser ; ne pas divulguer d'infos sensibles.
Bornes systématiques	`fgets/snprintf/strncpy` ; bannir `gets/strcpy` — cf. U2.

2. L'analyse statique (SAST)

Un outil **SAST** examine le code source *sans l'exécuter* pour repérer des motifs à risque : fonctions dangereuses, concaténations vers un `execute`, `shell=True`, etc. Avantage : tôt dans le cycle, couverture large ; inconvénient : **faux positifs** (alerte sans bug réel) et **faux négatifs** (bug manqué). Le SAST se combine avec la revue de code, les sanitizers (U5) et le fuzzing (U11).

3. Outils

Outil	Cible	Détecte (exemples)
`bandit`	Python	`shell=True`, SQL concaténé, désérialisation, secrets.
`cppcheck`	C/C++	double-free, fuites, hors-bornes, variables non initialisées.
`semgrep`	multi-langages	règles personnalisables (motifs source→sink).

Exemple réel — `bandit` sur l'application web de l'Unité 7/8 :

BANDIT — SORTIE RÉELLE (APP.PY VULNÉRABLE)

```
[B602:subprocess_popen_with_shell_equals_true]  Severity: High   Confidence: High
    Location: app.py:24   (subprocess avec shell=True)
[B608:hardcoded_sql_expressions]  Severity: Medium  Confidence: Low
    Location: app.py:17   (requete SQL construite par concatenation)
[B404:blacklist]  Severity: Low  (import subprocess : a surveiller)
```

Exemple réel — `cppcheck` sur le code de l'Unité 3 :

CPPCHECK — SORTIE RÉELLE

```
double_free.c:8:5: error: Memory pointed to by 'p' is freed twice. [doubleFree]
    free(p);   /* BUG : double-free */
```

4. Trier les résultats

Un rapport SAST se **trie** : confirmer les vrais positifs (sévérité/confiance), écarter les faux positifs avec justification, prioriser par gravité (CVSS, U10). Ne jamais « éteindre » une règle sans l'avoir comprise.

5. Points clés

- Prévenir coûte moins que corriger : valider, séparer, encoder, moindre privilège.
- Le SAST détecte tôt mais produit faux positifs/négatifs : il se trie et se combine.
- bandit (Python), cppcheck (C), semgrep (règles) : intégrables en CI.

Programmation sûre évoquée dans meltypot ; outils SAST auteur ; sorties bandit/cppcheck vérifiées.